

AULA 14

Complexidade de Algoritmos

Rastreamento de execução das funções que aparecem em Aula14_Teoria.pptx

Laço simples ("crescimento linear")

function soma(v) — qual é a resultante para um vetor genérico?

```
function soma(v)
  s = 0
  for x in v
    s += x
  end
  return s
end
```

s → parte de 0

para cada x em v → s acumula s + x (uma soma por elemento – não há valor concreto de v neste slide, então o traço é estrutural)

soma(v) → retorna s = soma de todos os elementos de v, ao final do laço

Fibonacci recursivo ingênuo

function fib(n) — árvore de chamadas para fib(5), como no slide "Árvore de chamadas"

```
function fib(n)
  if n <= 1
    return n
  end
  return fib(n-1) + fib(n-2)
end
```

chamada: fib(5)

fib(5) → = fib(4) + fib(3)

fib(4) → = fib(3) + fib(2)

fib(3) (chamada 2×) → = fib(2) + fib(1), em cada uma das duas vezes

fib(2) (chamada 3×) → = fib(1) + fib(0), em cada uma das três vezes

fib(1) (caso-base, chamado 5×) → retorna 1 diretamente, sem nova chamada

fib(0) (caso-base, chamado 3×) → retorna 0 diretamente, sem nova chamada

total de chamadas à função fib → 15 (1×fib(5) + 1×fib(4) + 2×fib(3) + 3×fib(2) + 5×fib(1) + 3×fib(0))

fib(5) → 5 (resultado final, depois de somar toda a árvore)

Fibonacci iterativo

function fib_iter(n) — passo a passo para fib_iter(5)

```
function fib_iter(n)
  a, b = 0, 1
  for i in 2:n
    a, b = b, a + b
  end
  return n == 0 ? 0 : b
end
```

chamada: fib_iter(5)

a, b → começam em 0, 1

i = 2 → a, b = 1, 0+1 → a=1, b=1

i = 3 → a, b = 1, 1+1 → a=1, b=2

i = 4 → a, b = 2, 1+2 → a=2, b=3

i = 5 → a, b = 3, 2+3 → a=3, b=5

fib_iter(5) → 5 (n ≠ 0, então retorna b)

Experimento 1: laços

ops_linear, ops_quadratico, ops_cubico — para $n = 20$ (valor padrão do slider @bind n Slider(1:100, default=20))

```
ops_linear(n) = n
ops_quadratico(n) = n^2
ops_cubico(n) = n^3

[n ops_linear(n) ops_quadratico(n) ops_cubico(n)]
```

```
n → 20 (valor padrão do slider)
ops_linear(20) → 20
ops_quadratico(20) →  $20^2 = 400$ 
ops_cubico(20) →  $20^3 = 8000$ 
resultado (vetor-linha) → [20 20 400 8000]
```

Experimento 3: Fibonacci (contagem de chamadas)

calls_naive(n) — quantas chamadas a recorrência prevê, para $n = 5$

```
calls_naive(n) = n <= 1 ? 1 : calls_naive(n-1) + calls_naive(n-2) + 1

# chamada: calls_naive(5)
```

```
calls_naive(0) → 1 (caso-base)
calls_naive(1) → 1 (caso-base)
calls_naive(2) → calls_naive(1) + calls_naive(0) + 1 = 1 + 1 + 1 = 3
calls_naive(3) → calls_naive(2) + calls_naive(1) + 1 = 3 + 1 + 1 = 5
calls_naive(4) → calls_naive(3) + calls_naive(2) + 1 = 5 + 3 + 1 = 9
calls_naive(5) → calls_naive(4) + calls_naive(3) + 1 = 9 + 5 + 1 = 15 (bate exatamente
com o total de chamadas contado na árvore de fib(5) acima)
```

Experimento 4: custo marginal

delta(T, n) — para as funções linear e quadratica, em $n = 4$ (a mesma linha da tabela "Custo local estável"/"Custo local crescente")

```
delta(T, n) = T(n) - T(n-1)
```

```
linear(n) = 3n + 5
quadratica(n) = n^2
```

```
delta(linear, 4)
delta(quadratica, 4)
```

```
delta(T, n) → recebe uma função T como argumento e calcula T(n) - T(n-1)
linear(4) →  $3 \times 4 + 5 = 17$ 
linear(3) →  $3 \times 3 + 5 = 14$ 
delta(linear, 4) →  $17 - 14 = 3$  (confere com  $\Delta T(n)=3$  da tabela "T(n)=3n+5")
quadratica(4) →  $4^2 = 16$ 
quadratica(3) →  $3^2 = 9$ 
delta(quadratica, 4) →  $16 - 9 = 7$  (confere com  $\Delta T(n)=7$  da linha n=4 da tabela "T(n)=n^2")
```